



Golang Gin Backend 11

Integrasi **GORM** dan **PostgreSQL** untuk backend Go Gin: model, migrasi, CRUD, relasi, dan transaksi.



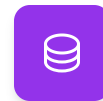
Juara Coding

June 2026

Tujuan Pembelajaran



Peserta memahami GORM di Go, integrasi Gin ke PostgreSQL, model, migrasi, CRUD, relasi data, dan transaction.



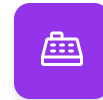
Integrasi GORM dan Database

Menghubungkan aplikasi Gin dengan PostgreSQL menggunakan GORM secara tepat dan terstruktur.



Model, Migrasi, dan CRUD

Mendefinisikan model, menjalankan migrasi otomatis, dan mengimplementasikan operasi CRUD.



Relasi dan Transaction

Mengelola relasi data serta memakai transaction untuk menjaga konsistensi operasi.



Pengenalan GORM dan PostgreSQL

GORM memudahkan pemetaan struct Go ke tabel database. **PostgreSQL** stabil, kaya fitur, dan cocok untuk backend API yang terstruktur.

Instalasi Dependency



Package utama

Instal paket **Gin**, **GORM**, dan driver **PostgreSQL** untuk memulai backend.



Manfaat pemisahan

Struktur ini membuat kode lebih mudah diuji dan dikembangkan lebih lanjut.



Struktur backend

Pisahkan file **main**, konfigurasi database, model, handler, dan route agar rapi.

```
▼ cmd/  
  └─ main.go  
▼ internal/  
  └─ handlers/  
    └─ models/  
      └─ services/  
▼ pkg/  
  └─ utils/  
    └─ middleware/  
  └─ config/  
    └─ migrations/  
  └─ docs/  
  └─ go.mod  
  └─ go.sum  
  └─ Makefile
```

Koneksi GORM ke PostgreSQL

Import Packages:

```
import {  
  "gorm.io/gorm"  
  "gorm.io/driver/postgres"  
}
```

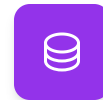


Connect to Database:

```
dsn := "host=localhost user=myuser password=mypassword dbname=mydb  
port=5432 sslmode=disable"  
  
db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{})  
if err := nil {  
  log.Fatal("Failed to connect to database:", err)  
}
```



Koneksi database memakai driver PostgreSQL di GORM dengan DSN, lalu instance disimpan untuk dipakai model, service, dan handler.



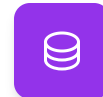
DSN PostgreSQL

DSN memuat host, user, password, dbname, port, sslmode, dan timezone untuk koneksi.



Inisialisasi GORM

Setelah koneksi berhasil, instance database dibuat melalui driver PostgreSQL pada GORM.



Pemakaian Instance

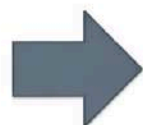
Instance database disimpan agar dapat digunakan oleh model, service, dan handler aplikasi.

Contoh Koneksi Database

Bagian	Kode
Import	<code>"gorm.io/driver/postgres"; "gorm.io/gorm"</code>
DSN	<code>host=localhost user=postgres password=postgres</code>
Open DB	<code>gorm.Open(postgres.Open(dsn), &gorm.Config{})</code>
Error Handling	<code>panic("gagal koneksi database")</code>

Contoh ini menunjukkan cara membuka koneksi PostgreSQL dengan *GORM* menggunakan *DSN*, lalu menangani error jika koneksi gagal.

users		
	VARCHAR(100)	
name:	VARCHAR(100)	
	VARCHAR(150)	
email:	VARCHAR(150)	
created_at:	TIMESTAMP	



```
CREATE TABLE users {
  ID INT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(150),
  created_at TIMESTAMP
}
```

```
type User struct {
  ID int "db:"id""
  Username string "db:"username""
  Email string "db:"email""
  CreatedAt time.Time "db:"created_at""
}
```

Konsep Model di GORM

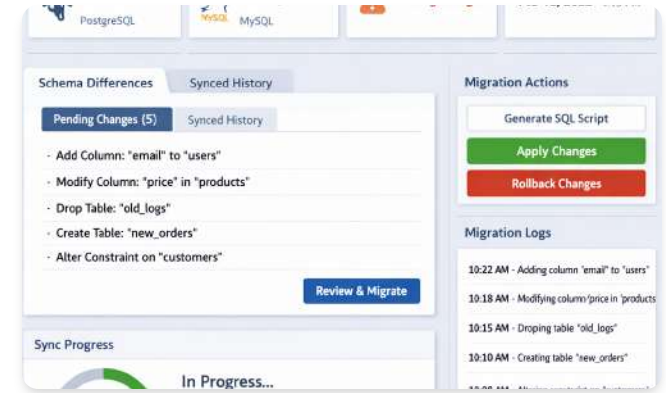
Model GORM direpresentasikan sebagai struct Go untuk memetakan field ke kolom tabel dengan aturan dan relasi data yang konsisten.

Contoh Model Dasar

Field	Tipe	Keterangan
ID	uint	Primary key
Name	string	Maks. 100, wajib diisi
Email	string	Unik, maks. 100, wajib diisi
CreatedAt	time.Time	Waktu data dibuat
UpdatedAt	time.Time	Waktu data diperbarui

Model ini membentuk tabel users dengan kolom inti yang umum dipakai pada aplikasi backend menggunakan GORM.

Auto Migration



01 Membuat atau menyesuaikan tabel

Auto migration digunakan untuk membuat atau menyesuaikan tabel sesuai model yang telah didefinisikan.

02 Mempermudah sinkronisasi database

Fitur ini membantu menjaga struktur database tetap sinkron selama proses pengembangan aplikasi.

03 Perlu hati-hati pada perubahan besar

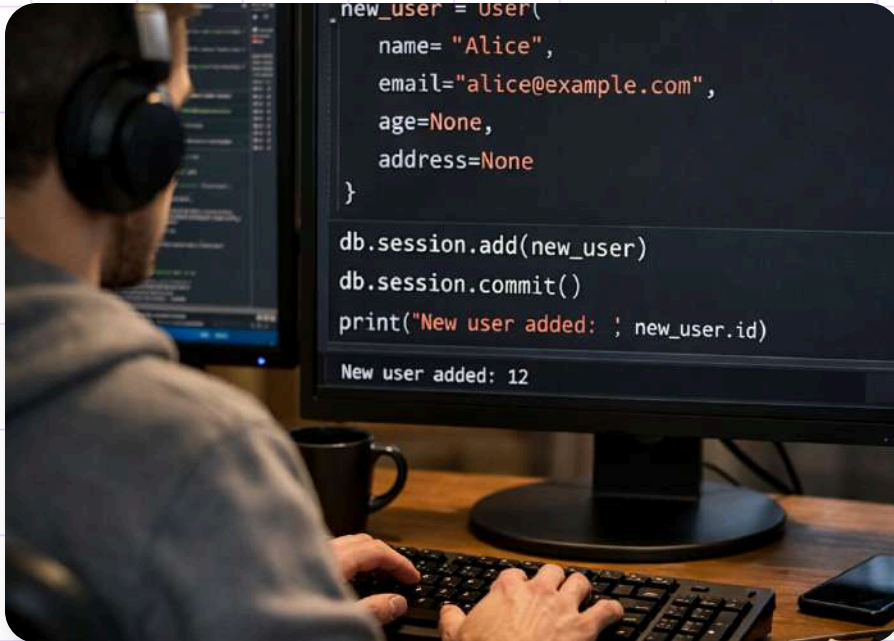
Perubahan skema yang besar tetap perlu dikontrol agar tidak memicu konflik dengan data yang sudah ada.

Contoh Auto Migration

Komponen	Keterangan
db.AutoMigrate	Menjalankan migrasi otomatis berdasarkan model
User, Product, Order	Model yang digunakan untuk membuat tabel
Jika tabel belum ada	Tabel akan dibuat otomatis
Jika struktur berubah	Struktur dasar disesuaikan
Jika err != nil	Aplikasi memunculkan panic migration

Perintah ini membuat tabel yang belum ada dan menyesuaikan struktur dasar sesuai model yang digunakan pada aplikasi.

CRUD: Create



Operasi create digunakan untuk menambahkan data baru ke database dengan method Create pada GORM.



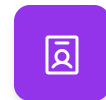
Tambah Data Baru

Method Create menyimpan satu objek baru ke database secara langsung.



Simpan Banyak Objek

GORM juga mendukung penyimpanan banyak objek sekaligus dalam satu proses.



ID Terisi Otomatis

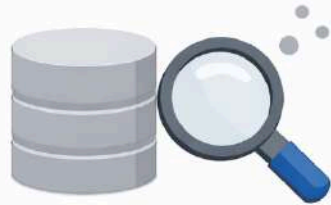
Setelah insert berhasil, field seperti ID biasanya diisi otomatis oleh PostgreSQL.

Contoh Create Data

Langkah	Kode/Objek
Inisialisasi user	<pre>user := User{Name: "Andi", Email: "andi@mail.com"}</pre>
Simpan ke database	<pre>result := db.Create(&user)</pre>
Cek error	<pre>if result.Error != nil { panic(result.Error) }</pre>
Gunakan ID baru	<pre>user.ID tersedia setelah create</pre>

Setelah data berhasil dibuat, nilai **user.ID** dapat langsung digunakan untuk relasi data atau sebagai bagian dari respons API.

CRUD: Read

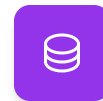


No Data Available

Your query returned no results.

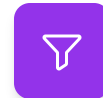
Run New Query

Operasi read mengambil data dari database dengan filter, sorting, limit, dan preload relasi untuk endpoint daftar dan detail.



Ambil Satu atau Banyak Data

GORM mendukung pengambilan satu record maupun banyak record sesuai kebutuhan endpoint.



Filter dan Sorting

Gunakan Where dan sorting untuk menampilkan data yang relevan dan terurut.



Limit dan Preload

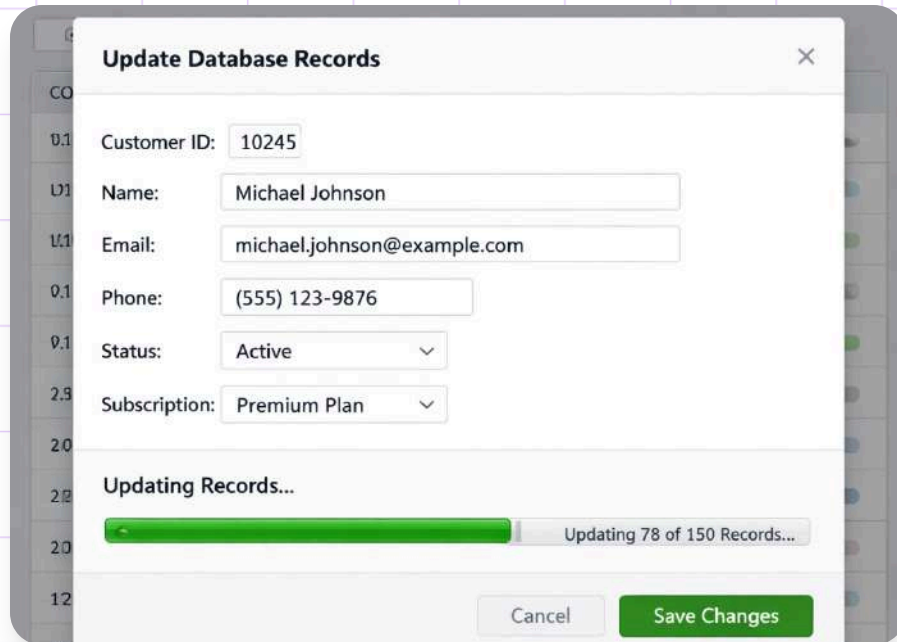
Batasi jumlah data dan muat relasi agar respons backend lebih lengkap.

Contoh Read Data

Operasi	Kode	Keterangan
Ambil semua data	<code>db.Find(&users)</code>	Mengambil seluruh data user
Ambil satu data by ID	<code>db.First(&user, 1)</code>	Mengambil user dengan ID 1
Filter by email	<code>Where(...).First</code>	Mengambil user berdasarkan email

Tabel ini merangkum query dasar untuk mengambil seluruh data, satu data berdasarkan ID, dan data dengan filter email.

CRUD: Update



Update Database Records

Customer ID: 10245

Name: Michael Johnson

Email: michael.johnson@example.com

Phone: (555) 123-9876

Status: Active

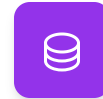
Subscription: Premium Plan

Updating Records...

Updating 78 of 150 Records...

Cancel Save Changes

Operasi update mengubah data yang sudah ada, baik pada field tertentu maupun seluruh model secara terarah.



Ubah Data yang Ada

Update dipakai untuk memodifikasi record yang sudah tersimpan di basis data.



Field atau Seluruh Model

GORM mendukung pembaruan sebagian field maupun semua field pada model.



Perubahan yang Terarah

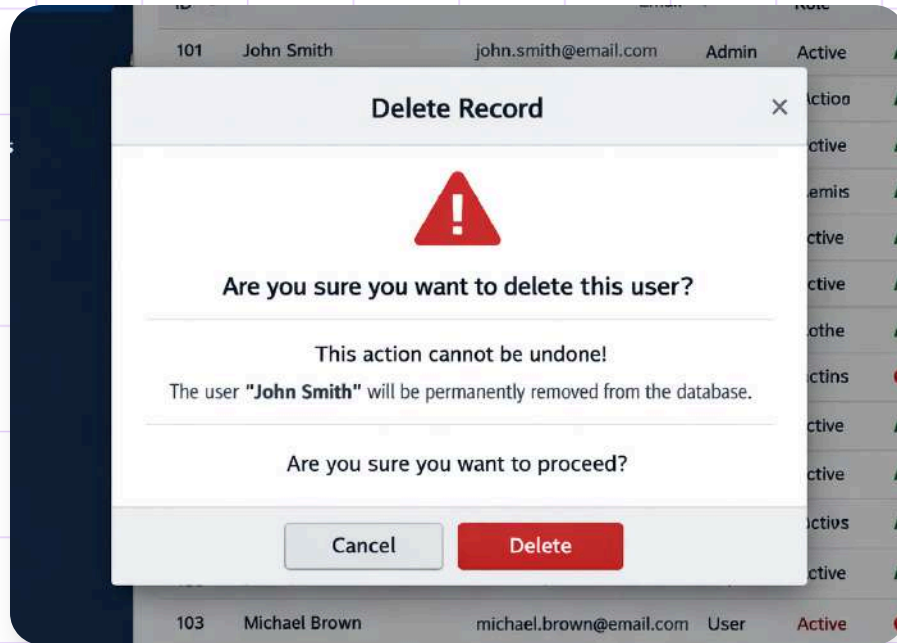
Update harus fokus pada atribut yang benar-benar dibutuhkan oleh logika bisnis aplikasi.

Contoh Update Data

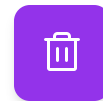
Method	Fungsi	Contoh
Update	Ubah satu field	Update("Name", "Andi")
Updates	Ubah beberapa field	Updates(Name, Email)

Update cocok untuk satu field, sedangkan **Updates** berguna saat mengubah beberapa field sekaligus.

CRUD: Delete



Operasi delete menghapus data berdasarkan kondisi tertentu, baik lewat objek maupun ID, dengan pertimbangan relasi dan histori transaksi.



Hapus dengan kondisi

Data dihapus dari database sesuai kondisi yang ditentukan pada query.



Berdasarkan objek atau ID

GORM mendukung penghapusan menggunakan objek model atau langsung dengan ID.



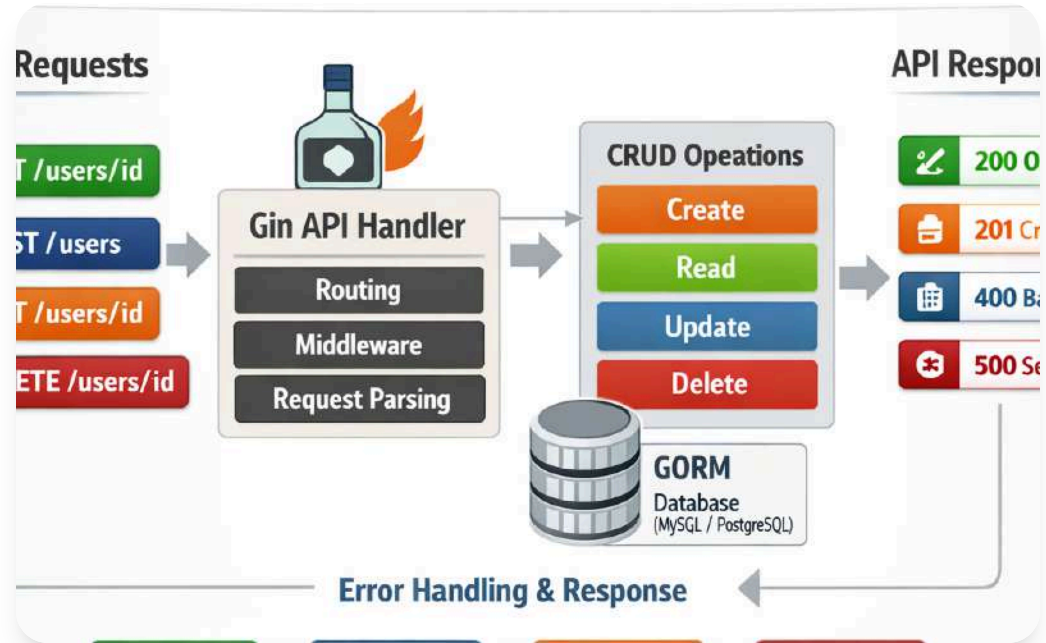
Perlu kehati-hatian

Penghapusan dapat memengaruhi relasi data dan histori transaksi dalam sistem.

Contoh Delete Data

Metode	Kode	Keterangan
Cari lalu hapus	<code>db.First(&user, 1)</code> <code>db.Delete(&user)</code>	Temukan data, lalu hapus.
Hapus via ID	<code>db.Delete(&User{}, 2)</code>	Hapus langsung pakai ID.

Penghapusan data dapat dilakukan setelah data ditemukan lebih dulu atau langsung menggunakan ID target yang ingin dihapus.



Integrasi CRUD dengan Gin

Handler Gin memproses request, validasi input, operasi database via GORM, lalu mengembalikan respons JSON secara terstruktur.

Association di GORM



Relasi Antar Model

Association menghubungkan model seperti user, order, product, dan category di backend.



Cerminkan Relasi Nyata

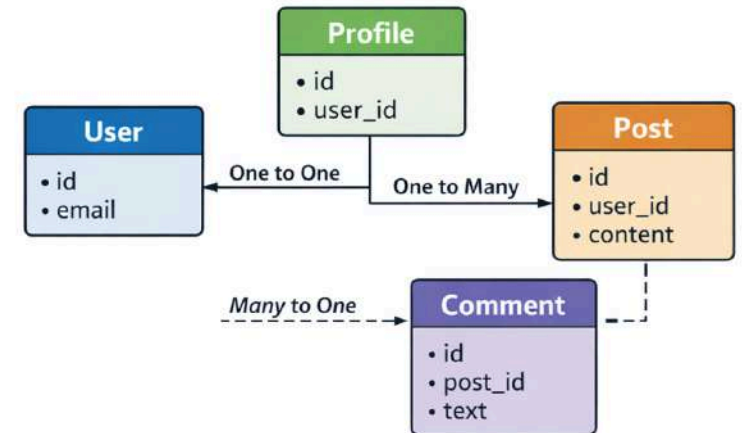
Membantu struktur data merepresentasikan hubungan nyata antar entitas bisnis.



Jenis Association

Mencakup pola umum seperti one to one, one to many, dan many to many.

Backend Model Relations



Contoh One to Many

Struct	Field	Type	Keterangan
User	ID	uint	Primary key user
User	Orders	[]Order	Satu user punya banyak order
Order	UserID	uint	Foreign key ke user
Order	Total	int	Nilai total order

Tabel ini merangkum relasi *one-to-many* pada contoh Go, di mana **satu user** dapat memiliki banyak order dan setiap order terhubung ke **satu user**.

Mengambil Data dengan Preload

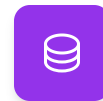
```
users []User

== db.Preload("Orders.OrderItems").Preload("Orders")
    Find(&users).Error

err := nil {
    log.Println("Error:", err)
    return
}

// Use the loaded data
Println(users)
```

GORM menyediakan *Preload* untuk mengambil data beserta relasinya agar hasil query lebih lengkap dan efisien.



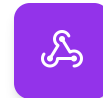
Relasi Sekaligus

Preload memuat data utama dan relasinya dalam satu alur query yang rapi.



Kurangi Query Manual

Tidak perlu memanggil query tambahan untuk setiap relasi yang ingin ditampilkan.



Cocok untuk Endpoint

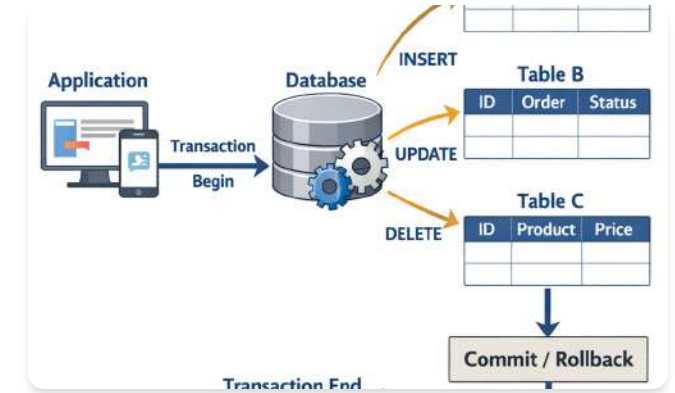
Sangat membantu saat membuat endpoint detail atau daftar data bertingkat.

Contoh Preload Association

Kode	Fungsi
<code>db.Preload("Orders")</code>	Memuat relasi order terkait
<code>Find(&users)</code>	Mengambil data user beserta relasinya

Preload membuat setiap user disertai daftar order terkait, sehingga cocok untuk endpoint yang memerlukan data induk dan anak dalam satu respons.

Transaction di GORM



01 Menjaga konsistensi data

Transaction memastikan beberapa operasi database diproses sebagai satu unit yang utuh dan konsisten.

03 Penting untuk proses kompleks

Sangat berguna pada proses lintas tabel, seperti pembuatan order dan pengurangan stok produk.

02 Rollback saat terjadi gagal

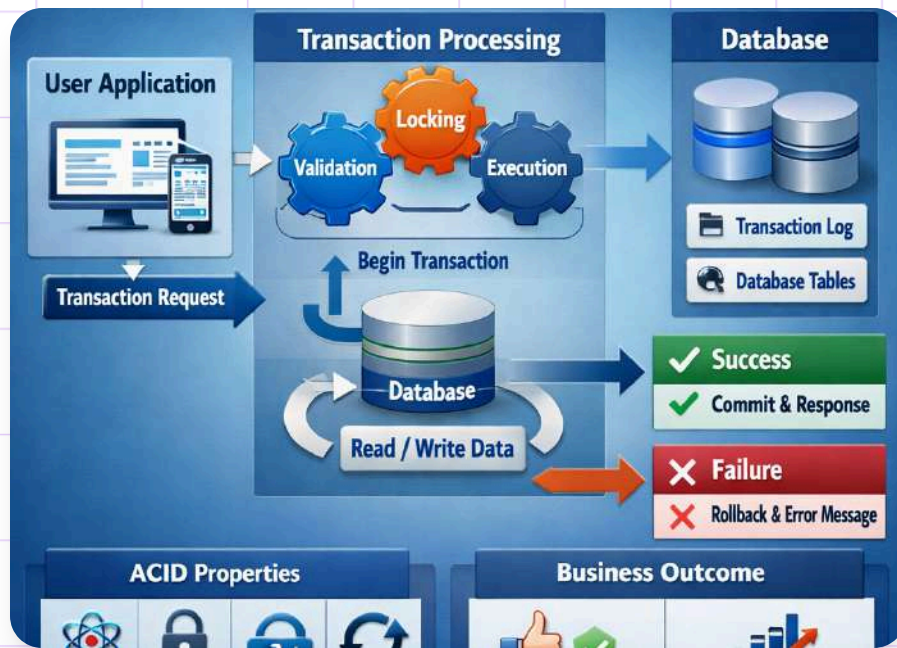
Jika satu langkah gagal, seluruh perubahan dapat dibatalkan agar data tidak setengah tersimpan.

Contoh Transaction

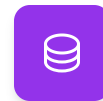
Langkah	Operasi
1	Create order
2	Update stock
3	Rollback otomatis jika gagal

Transaksi memastikan beberapa operasi database berjalan sebagai satu kesatuan. Jika satu proses gagal, semua perubahan akan dibatalkan otomatis.

Kapan Menggunakan Transaction

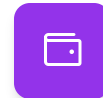


Gunakan transaction saat beberapa operasi database saling bergantung agar data tetap konsisten dan tidak tersimpan parsial.



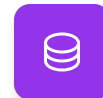
Operasi Saling Bergantung

Cocok untuk proses yang melibatkan beberapa query yang harus berhasil bersama.



Kasus Penggunaan Umum

Contohnya pembayaran, checkout, transfer saldo, dan pembaruan stok barang.



Mencegah Data Parsial

Tanpa transaction, data antar tabel bisa tidak sinkron dan lebih sulit diperbaiki.

Praktik Baik GORM

01
10

Pisahkan model dan handler

Struktur terpisah membuat kode lebih rapi, mudah diuji, dan mudah dipelihara.



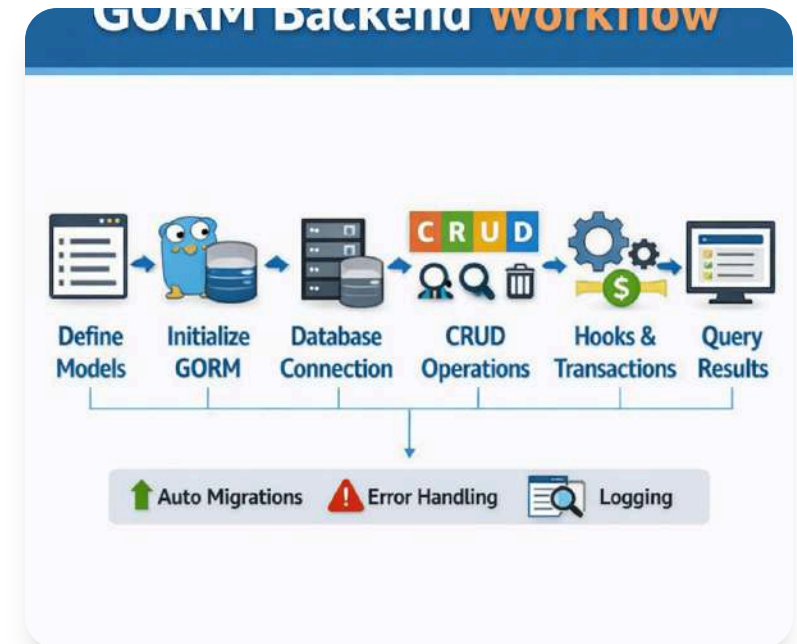
Gunakan transaction

Transaction penting untuk proses kritis agar data tetap konsisten dan lebih aman.

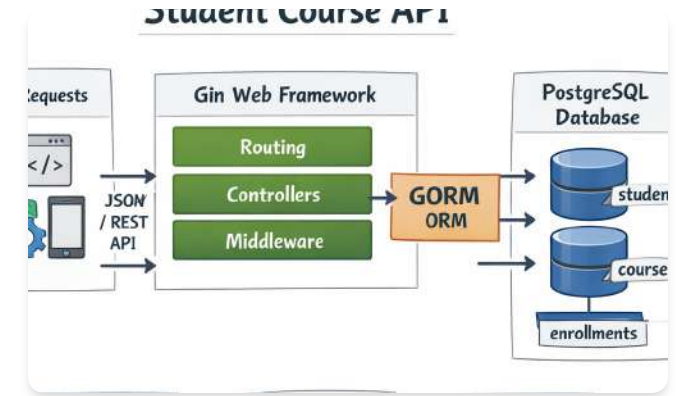


Validasi dan query efisien

Validasi input sebelum query, gunakan Preload hanya saat perlu, dan kelola migration secara terkontrol.



Latihan Praktik 1



01 Stack dan tujuan

Buat API sederhana untuk entitas Student dan Course dengan Gin, GORM, dan PostgreSQL.

03 Relasi dan query

Tetapkan satu Course memiliki banyak Student, lalu tampilkan Course beserta daftar Student dengan Preload.

02 Implementasi utama

Susun model, jalankan auto migration, lalu lengkapi operasi CRUD untuk Student.

Latihan Praktik 2



01 Entitas Utama

Buat entitas User, Product, dan Order sebagai dasar sistem order sederhana.

03 Rollback dan Detail

Batalkan proses jika stok kosong atau update gagal, lalu tampilkan detail order beserta user dan product.

02 Transaksi Order

Simpan order dan kurangi stok product dalam satu transaction saat order dibuat.

Ringkasan Materi

No Backend Code,
&
No Database

*"Penggunaan **GORM** dengan **PostgreSQL** dalam backend **Gin** mencakup koneksi database, model, auto migration, **CRUD**, association, dan transaction."*

Kesimpulan Sesi